

✓ University of Utah CS 6340/5340 NLP Fall 2024 Assignment 5

First, make a copy of this notebook: File > Save a copy in Drive.

For Part 4 and 5: Connect to a GPU by clicking on the arrow next to "Connect" in the upper right corner, then click "Change runtime type" and select "T4 GPU".

Turn off the code completion by going to Tools > Settings > Editor > Automatically trigger code completions > Uncheck the box next to it. **Using the completed code is the same as copying it which is the academic misconduct in this class.**

Run the cells with your solutions such that the output is visible. When you are ready to submit your solution to Gradescope, do the following: File > Print > Destination: Save as PDF > Save. Upload the pdf to Gradescope.

✓ Text Generation Evaluation (100 points + 7 bonus points)

In this assignment, you will investigate evaluation of text generation in the context of summarization.

You will use the data released with the following publication: [Fabbri et al. \(2021\). SummEval: Re-evaluating Summarization Evaluation.](#) This data is available in HuggingFace: <https://huggingface.co/datasets/mteb/summeval> under the name `mteb/summeval`. It contains 16 generated and 11 human-authored summaries for each of 100 source documents.

You will compute BLEU, ROUGE-2, BERTScore, and BLEURT using existing implementations on this data and measure the correlation of scores you get with these automatic metrics with human judgments of relevance, coherence, fluency, and consistency.

In the bonus part, you have an opportunity to earn extra points by additionally evaluating LLM-as-Judge.

Notes:

1. The main challenge in this homework is to understand how to use the existing implementations by reading their documentation, particularly to determine the expected input format for each function. Once you complete the BLEU evaluation, the process will become repetitive for the other metrics.
2. *You are welcome (but not required) to experiment with different argument settings for the existing functions to explore their impact on the results.* For example, use smoothing for BLEU or not, etc.
3. You are **not** allowed to directly prompt LLMs to ask how to use the existing implementations. The point is that you learn how to read this kind of code and figure this on your own.

✓ Part 1: Data Processing (20 points)

You **must** use Huggingface's [datasets](#) and its functions to load the `mteb/summeval` dataset. When loaded, the dataset will be of the type `DatasetDict`. You are required to use this data structure for subsequent processing steps.

You can explore the dataset using the Dataset Viewer on its [huggingface page](#). Each source document (a row in the Dataset Viewer) contains the following information:

- A column with a list of 16 generated summaries,
- A column with a list of 11 human-authored summaries,
- Columns with lists of 16 human-authored scores for relevance, coherence, consistency, and fluency.

Most of the existing automatic evaluation measurements accept a generated summary with a **list** of human-authored reference summaries.

Your task is to process the `DatasetDict` to produce:

- A list of 1600 generated summaries: Flatten the column containing 100 lists of 16 generated summaries.
- A list of 1600 lists of 11 human-authored summaries: Each of these 1600 lists should correspond to one generated summary. The first 16 generated summaries will share the same 11 reference summaries, the next 16 will share another set of 11 reference summaries, and so on.
- Lists of 1600 human-authored scores for relevance, coherence, consistency, and fluency: Create one list for each type of score by flattening the corresponding columns.

TIP:

- Use assertion to check that the lists are of the right length.

Install `datasets` with the following command.

```
!pip install datasets
```

```
Collecting datasets
  Downloading datasets-3.1.0-py3-none-any.whl.metadata (20 kB)
Requirement already satisfied: filelock in /usr/local/lib/python3.10/dist-packages (from datasets) (3.16.1)
Requirement already satisfied: numpy>=1.17 in /usr/local/lib/python3.10/dist-packages (from datasets) (1.26.4)
Requirement already satisfied: pyarrow>=15.0.0 in /usr/local/lib/python3.10/dist-packages (from datasets) (17.0.0)
Collecting dill<0.3.9,>=0.3.0 (from datasets)
  Downloading dill-0.3.8-py3-none-any.whl.metadata (10 kB)
Requirement already satisfied: pandas in /usr/local/lib/python3.10/dist-packages (from datasets) (2.2.2)
Requirement already satisfied: requests>=2.32.2 in /usr/local/lib/python3.10/dist-packages (from datasets) (2.32.3)
Requirement already satisfied: tqdm>=4.66.3 in /usr/local/lib/python3.10/dist-packages (from datasets) (4.66.6)
Collecting xxhash (from datasets)
  Downloading xxhash-3.5.0-cp310-cp310-manylinux_2_17_x86_64.manylinux2014_x86_64.whl.metadata (12 kB)
Collecting multiprocess<0.70.17 (from datasets)
  Downloading multiprocess-0.70.16-py310-none-any.whl.metadata (7.2 kB)
Collecting fsspec<=2024.9.0,>=2023.1.0 (from fsspec[http]<=2024.9.0,>=2023.1.0->datasets)
  Downloading fsspec-2024.9.0-py3-none-any.whl.metadata (11 kB)
Requirement already satisfied: aiohttp in /usr/local/lib/python3.10/dist-packages (from datasets) (3.11.9)
Requirement already satisfied: huggingface-hub>=0.23.0 in /usr/local/lib/python3.10/dist-packages (from datasets) (0.26.3)
Requirement already satisfied: packaging in /usr/local/lib/python3.10/dist-packages (from datasets) (24.2)
Requirement already satisfied: pyyaml>=5.1 in /usr/local/lib/python3.10/dist-packages (from datasets) (6.0.2)
Requirement already satisfied: aiohappyeyeballs>=2.3.0 in /usr/local/lib/python3.10/dist-packages (from aiohttp->datasets) (2.4.4)
Requirement already satisfied: aiosignal>=1.1.2 in /usr/local/lib/python3.10/dist-packages (from aiohttp->datasets) (1.3.1)
Requirement already satisfied: async-timeout<6.0,>=4.0 in /usr/local/lib/python3.10/dist-packages (from aiohttp->datasets) (4.0.3)
Requirement already satisfied: attrs>=17.3.0 in /usr/local/lib/python3.10/dist-packages (from aiohttp->datasets) (24.2.0)
Requirement already satisfied: frozenlist>=1.1.1 in /usr/local/lib/python3.10/dist-packages (from aiohttp->datasets) (1.5.0)
Requirement already satisfied: multidict<7.0,>=4.5 in /usr/local/lib/python3.10/dist-packages (from aiohttp->datasets) (6.1.0)
Requirement already satisfied: propcache>=0.2.0 in /usr/local/lib/python3.10/dist-packages (from aiohttp->datasets) (0.2.1)
Requirement already satisfied: yarl<2.0,>=1.17.0 in /usr/local/lib/python3.10/dist-packages (from aiohttp->datasets) (1.18.3)
Requirement already satisfied: typing-extensions>=3.7.4.3 in /usr/local/lib/python3.10/dist-packages (from huggingface-hub>=0.23.0->datasets) (4.12.2)
Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.10/dist-packages (from requests>=2.32.2->datasets) (3.3.2)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.10/dist-packages (from requests>=2.32.2->datasets) (3.10.1)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.10/dist-packages (from requests>=2.32.2->datasets) (2.2.3)
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.10/dist-packages (from requests>=2.32.2->datasets) (2024.7.4)
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.10/dist-packages (from pandas->datasets) (2.9.0)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.10/dist-packages (from pandas->datasets) (2024.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.10/dist-packages (from pandas->datasets) (2024.2)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.10/dist-packages (from python-dateutil>=2.8.2->pandas->datasets) (1.17.0)
Downloading datasets-3.1.0-py3-none-any.whl (480 kB)
480.6/480.6 kB 18.2 MB/s eta 0:00:00
Downloading dill-0.3.8-py3-none-any.whl (116 kB)
116.3/116.3 kB 10.2 MB/s eta 0:00:00
Downloading fsspec-2024.9.0-py3-none-any.whl (179 kB)
179.3/179.3 kB 15.7 MB/s eta 0:00:00
Downloading multiprocess-0.70.16-py310-none-any.whl (134 kB)
134.8/134.8 kB 12.4 MB/s eta 0:00:00
Downloading xxhash-3.5.0-cp310-cp310-manylinux_2_17_x86_64.manylinux2014_x86_64.whl (194 kB)
194.1/194.1 kB 16.5 MB/s eta 0:00:00
Installing collected packages: xxhash, fsspec, dill, multiprocess, datasets
Attempting uninstall: fsspec
  Found existing installation: fsspec 2024.10.0
  Uninstalling fsspec-2024.10.0:
    Successfully uninstalled fsspec-2024.10.0
ERROR: pip's dependency resolver does not currently take into account all the packages that are installed. This behaviour is
  gcsfs 2024.10.0 requires fsspec==2024.10.0, but you have fsspec 2024.9.0 which is incompatible.
Successfully installed datasets-3.1.0 dill-0.3.8 fsspec-2024.9.0 multiprocess-0.70.16 xxhash-3.5.0
```

Your code should be written in the next code cell.

```
from datasets import load_dataset
dataset = load_dataset("mteb/summeval")
print(dataset)
```

```

/usr/local/lib/python3.10/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/tokens), set
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
  warnings.warn(
README.md: 100%                                635/635 [00:00<00:00, 13.2kB/s]
dataset_infos.json: 100%                        2.38k/2.38k [00:00<00:00, 93.0kB/s]
(...)00000-of-00001-35901af5f6649399.parquet: 100% 423k/423k [00:00<00:00, 4.45MB/s]
Generating test split: 100%                    100/100 [00:00<00:00, 876.97 examples/s]
DatasetDict({
  test: Dataset({
    features: ['machine_summaries', 'human_summaries', 'relevance', 'coherence', 'fluency', 'consistency', 'text', 'id']
    num_rows: 100
  })
})

```

```
print(dataset["test"][0])
```

```
{'machine_summaries': ['donald sterling , nba team last year . sterling 's wife sued for $ 2.6 million in gifts . sterling s
```

```
test_data = dataset["test"]
test_data
```

```
Dataset({
  features: ['machine_summaries', 'human_summaries', 'relevance', 'coherence', 'fluency', 'consistency', 'text', 'id'],
  num_rows: 100
})

```

```
machine_summaries=[]
for doc in test_data["machine_summaries"]:
    for summary in doc:
        machine_summaries.append(summary)
assert len(machine_summaries) == 1600, "Machine summaries should have 1600 items"
```

```
human_summaries=[]
for i in range(len(machine_summaries)):
    doc_index = i // 16
    human_summaries.append(test_data["human_summaries"][doc_index])
assert len(human_summaries) == 1600, "Human summaries should have 1600 items"
```

```
relevance_scores = []
coherence_scores = []
fluency_scores = []
consistency_scores = []
for doc in test_data:
    relevance_scores.extend(doc["relevance"])
    coherence_scores.extend(doc["coherence"])
    fluency_scores.extend(doc["fluency"])
    consistency_scores.extend(doc["consistency"])

assert len(relevance_scores) == 1600, "Relevance scores should have 1600 items"
assert len(coherence_scores) == 1600, "Coherence scores should have 1600 items"
assert len(fluency_scores) == 1600, "Fluency scores should have 1600 items"
assert len(consistency_scores) == 1600, "Consistency scores should have 1600 items"
```

```
len(machine_summaries)
```

```
1600
```

✓ Part 2: BLEU Evaluation and Correlation Analysis (20 points)

You will use NLTK's implementation of BLEU, specifically [sentence_bleu](#). Refer to the linked documentation to understand how to use `sentence_bleu` correctly. For each generated summary:

1. Pass the generated summary and its associated list of human-authored summaries to `sentence_bleu`.
2. Record the resulting BLEU score in a list.

Using the list of BLEU scores, compute the **Pearson** and **Spearman** correlation coefficients with each of the following human-authored scores:

- Relevance
- Coherence
- Fluency
- Consistency

Your final output should be **8 correlation coefficients**: 4 for Pearson and 4 for Spearman.

You will need to import `nltk` and download `punkt` and `punkt_tab`.

```
import nltk
from nltk.translate.bleu_score import sentence_bleu
nltk.download('punkt')
nltk.download('punkt_tab')
```

 [nltk_data] Downloading package punkt to /root/nltk_data...
 [nltk_data] Unzipping tokenizers/punkt.zip.
 [nltk_data] Downloading package punkt_tab to /root/nltk_data...
 [nltk_data] Unzipping tokenizers/punkt_tab.zip.
 True

Use `scipy`'s implementation to calculate the Pearson and Spearman correlation coefficients.

```
from scipy.stats import pearsonr, spearmanr
```

Your code should be written in the next code cell.

```
bleu_scores = []
for i, generated_summary in enumerate(machine_summaries):
    bleu_score = sentence_bleu(human_summaries[i], generated_summary)
    bleu_scores.append(bleu_score)

pearson_relevance, _ = pearsonr(bleu_scores, relevance_scores)
spearman_relevance, _ = spearmanr(bleu_scores, relevance_scores)

pearson_coherence, _ = pearsonr(bleu_scores, coherence_scores)
spearman_coherence, _ = spearmanr(bleu_scores, coherence_scores)

pearson_fluency, _ = pearsonr(bleu_scores, fluency_scores)
spearman_fluency, _ = spearmanr(bleu_scores, fluency_scores)

pearson_consistency, _ = pearsonr(bleu_scores, consistency_scores)
spearman_consistency, _ = spearmanr(bleu_scores, consistency_scores)

print("Pearson Correlations:")
print(f"Relevance: {pearson_relevance}")
print(f"Coherence: {pearson_coherence}")
print(f"Fluency: {pearson_fluency}")
print(f"Consistency: {pearson_consistency}")

print("\nSpearman Correlations:")
print(f"Relevance: {spearman_relevance}")
print(f"Coherence: {spearman_coherence}")
print(f"Fluency: {spearman_fluency}")
print(f"Consistency: {spearman_consistency}")
```

 Pearson Correlations:
 Relevance: 0.22766302928240473
 Coherence: 0.1332069110705339
 Fluency: 0.18343657400166397
 Consistency: 0.10396401081669852

Spearman Correlations:
 Relevance: 0.21252935359514283
 Coherence: 0.17622606666797652
 Fluency: 0.13975227941763513
 Consistency: 0.09478717043463185

In the next text cell write a brief comment analyzing the results. Discuss any insights you observe, such as which human-authored scores correlate most strongly with BLEU and potential reasons for these patterns.

Answer

We can clearly see that BLEU scores have a very weak positive correlations. But comparatively it is highest with relevance. This is because BLEU evaluates n-gram overlaps, which directly relate to content inclusion and keyword similarity.

✓ Part 3: ROUGE-2 Evaluation and Correlation Analysis (20 points)

You will use [this implementation](#) of ROUGE-2. Use the `pip install` command below to install it. For each generated summary:

1. Instantiate a ROUGE-2 scorer with `RougeScorer()` and the right arguments.
2. Pass the generated summary and its associated list of human-authored summaries to your ROUGE-2 scorer.
3. Record the resulting ROUGE-2 score in a list.

Using the list of ROUGE-2 scores, compute the **Pearson** and **Spearman** correlation coefficients with each of the following human-authored scores:

- Relevance
- Coherence
- Fluency
- Consistency

Your final output should be **8 correlation coefficients**: 4 for Pearson and 4 for Spearman.

Install `rouge_score` with the following command.

```
!pip install rouge_score
```

```
Collecting rouge_score
  Downloading rouge_score-0.1.2.tar.gz (17 kB)
  Preparing metadata (setup.py) ... done
Requirement already satisfied: absl-py in /usr/local/lib/python3.10/dist-packages (from rouge_score) (1.4.0)
Requirement already satisfied: nltk in /usr/local/lib/python3.10/dist-packages (from rouge_score) (3.9.1)
Requirement already satisfied: numpy in /usr/local/lib/python3.10/dist-packages (from rouge_score) (1.26.4)
Requirement already satisfied: six>=1.14.0 in /usr/local/lib/python3.10/dist-packages (from rouge_score) (1.16.0)
Requirement already satisfied: click in /usr/local/lib/python3.10/dist-packages (from nltk->rouge_score) (8.1.7)
Requirement already satisfied: joblib in /usr/local/lib/python3.10/dist-packages (from nltk->rouge_score) (1.4.2)
Requirement already satisfied: regex>=2021.8.3 in /usr/local/lib/python3.10/dist-packages (from nltk->rouge_score) (2024.9.1)
Requirement already satisfied: tqdm in /usr/local/lib/python3.10/dist-packages (from nltk->rouge_score) (4.66.6)
Building wheels for collected packages: rouge_score
  Building wheel for rouge_score (setup.py) ... done
  Created wheel for rouge_score: filename=rouge_score-0.1.2-py3-none-any.whl size=24935 sha256=bf29a8f38b1c5bcee038044ea56ae
  Stored in directory: /root/.cache/pip/wheels/5f/dd/89/461065a73be61a532ff8599a28e9beef17985c9e9c31e541b4
Successfully built rouge_score
Installing collected packages: rouge_score
Successfully installed rouge_score-0.1.2
```

Your code should be written in the next code cell.

```
from rouge_score import rouge_scorer
scorer = rouge_scorer.RougeScorer(['rouge2'], use_stemmer=True)
rouge2_scores = []

for i, generated_summary in enumerate(machine_summaries):
    scores = [scorer.score(generated_summary, ref) for ref in human_summaries[i]]
    recall_scores = [score['rouge2'].recall for score in scores]
    rouge2_scores.append(max(recall_scores))

pearson_relevance, _ = pearsonr(rouge2_scores, relevance_scores)
spearman_relevance, _ = spearmanr(rouge2_scores, relevance_scores)

pearson_coherence, _ = pearsonr(rouge2_scores, coherence_scores)
spearman_coherence, _ = spearmanr(rouge2_scores, coherence_scores)

pearson_fluency, _ = pearsonr(rouge2_scores, fluency_scores)
spearman_fluency, _ = spearmanr(rouge2_scores, fluency_scores)
```

```
pearson_consistency, _ = pearsonr(rouge2_scores, consistency_scores)
spearman_consistency, _ = spearmanr(rouge2_scores, consistency_scores)
```

```
print("Pearson Correlations:")
print(f"Relevance: {pearson_relevance}")
print(f"Coherence: {pearson_coherence}")
print(f"Fluency: {pearson_fluency}")
print(f"Consistency: {pearson_consistency}")
```

```
print("\nSpearman Correlations:")
print(f"Relevance: {spearman_relevance}")
print(f"Coherence: {spearman_coherence}")
print(f"Fluency: {spearman_fluency}")
print(f"Consistency: {spearman_consistency}")
```

```
➦ Pearson Correlations:
Relevance: 0.16178729036337977
Coherence: 0.1854716941519997
Fluency: 0.1368089952808596
Consistency: 0.14724918616406038
```

```
Spearman Correlations:
Relevance: 0.18740370050196045
Coherence: 0.17503832115535004
Fluency: 0.09718025182808304
Consistency: 0.12601527818493852
```

In the next text cell, write a brief comment analyzing the results. Discuss any insights you observe, such as which human-authored scores correlate most strongly with ROUGE-2 and possible reasons for these patterns. Additionally, compare ROUGE-2 with BLEU, highlighting differences and similarities based on how these two metrics are defined.

Answer

We can see that Relevance and Coherence has the highest correlation again. We can see that this result is similar to BLEU scores.

Similarity: ROUGE-2 focuses on bigram matches, while BLEU considers n-grams of varying sizes.

Difference BLEU computes a geometric mean of n-gram precision scores and applies penalty for overly short summaries. ROUGE-2 focuses solely on recall and is less useful of shorter summaries.

✓ Part 4: BERTScore Evaluation and Correlation Analysis (20 points)

Now you will move from lexical overlap measurements to similarity-based one. In this part, you will use [BERTScore](#), specifically [this implementation](#). Use the `pip install` command below to install it. For each generated summary:

1. Pass the generated summary and its associated list of human-authored summaries to the score.
2. Record the resulting BERTScore F1 in a list.

Using the list of BERTScore-F1 scores, compute the **Pearson** and **Spearman** correlation coefficients with each of the following human-authored scores:

- Relevance
- Coherence
- Fluency
- Consistency

Your final output should be **8 correlation coefficients**: 4 for Pearson and 4 for Spearman.

Note: This part will be slower. Remember why? It will be faster if you connect to a GPU.

Install the package with the following command.

```
!pip install bert_score
```

```
➦ Collecting bert_score
  Downloading bert_score-0.3.13-py3-none-any.whl.metadata (15 kB)
Requirement already satisfied: torch>=1.0.0 in /usr/local/lib/python3.10/dist-packages (from bert_score) (2.5.1+cu121)
Requirement already satisfied: pandas>=1.0.1 in /usr/local/lib/python3.10/dist-packages (from bert_score) (2.2.2)
Requirement already satisfied: transformers>=3.0.0 in /usr/local/lib/python3.10/dist-packages (from bert_score) (4.46.3)
```

```
Requirement already satisfied: numpy in /usr/local/lib/python3.10/dist-packages (from bert_score) (1.26.4)
Requirement already satisfied: requests in /usr/local/lib/python3.10/dist-packages (from bert_score) (2.32.3)
Requirement already satisfied: tqdm>=4.31.1 in /usr/local/lib/python3.10/dist-packages (from bert_score) (4.66.6)
Requirement already satisfied: matplotlib in /usr/local/lib/python3.10/dist-packages (from bert_score) (3.8.0)
Requirement already satisfied: packaging>=20.9 in /usr/local/lib/python3.10/dist-packages (from bert_score) (24.2)
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.10/dist-packages (from pandas>=1.0.1->bert_s
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.10/dist-packages (from pandas>=1.0.1->bert_score) (202
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.10/dist-packages (from pandas>=1.0.1->bert_score) (2
Requirement already satisfied: filelock in /usr/local/lib/python3.10/dist-packages (from torch>=1.0.0->bert_score) (3.16.1)
Requirement already satisfied: typing-extensions>=4.8.0 in /usr/local/lib/python3.10/dist-packages (from torch>=1.0.0->bert_
Requirement already satisfied: networkx in /usr/local/lib/python3.10/dist-packages (from torch>=1.0.0->bert_score) (3.4.2)
Requirement already satisfied: Jinja2 in /usr/local/lib/python3.10/dist-packages (from torch>=1.0.0->bert_score) (3.1.4)
Requirement already satisfied: fsspec in /usr/local/lib/python3.10/dist-packages (from torch>=1.0.0->bert_score) (2024.9.0)
Requirement already satisfied: sympy==1.13.1 in /usr/local/lib/python3.10/dist-packages (from torch>=1.0.0->bert_score) (1.1
Requirement already satisfied: mpmath<1.4, >=1.1.0 in /usr/local/lib/python3.10/dist-packages (from sympy==1.13.1->torch>=1.0
Requirement already satisfied: huggingface-hub<1.0, >=0.23.2 in /usr/local/lib/python3.10/dist-packages (from transformers>=3
Requirement already satisfied: pyyaml>=5.1 in /usr/local/lib/python3.10/dist-packages (from transformers>=3.0.0->bert_score)
Requirement already satisfied: regex!=2019.12.17 in /usr/local/lib/python3.10/dist-packages (from transformers>=3.0.0->bert_
Requirement already satisfied: tokenizers<0.21, >=0.20 in /usr/local/lib/python3.10/dist-packages (from transformers>=3.0.0->
Requirement already satisfied: safetensors>=0.4.1 in /usr/local/lib/python3.10/dist-packages (from transformers>=3.0.0->bert
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.10/dist-packages (from matplotlib->bert_score) (1.
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.10/dist-packages (from matplotlib->bert_score) (0.12.1
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.10/dist-packages (from matplotlib->bert_score) (4
Requirement already satisfied: kiwisolver>=1.0.1 in /usr/local/lib/python3.10/dist-packages (from matplotlib->bert_score) (1
Requirement already satisfied: pillow>=6.2.0 in /usr/local/lib/python3.10/dist-packages (from matplotlib->bert_score) (11.0.
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.10/dist-packages (from matplotlib->bert_score) (3.
Requirement already satisfied: charset-normalizer<4, >=2 in /usr/local/lib/python3.10/dist-packages (from requests->bert_scor
Requirement already satisfied: idna<4, >=2.5 in /usr/local/lib/python3.10/dist-packages (from requests->bert_score) (3.10)
Requirement already satisfied: urllib3<3, >=1.21.1 in /usr/local/lib/python3.10/dist-packages (from requests->bert_score) (2.
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.10/dist-packages (from requests->bert_score) (20
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.10/dist-packages (from python-dateutil>=2.8.2->pandas>=1.0
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.10/dist-packages (from Jinja2->torch>=1.0.0->bert_s
Downloading bert_score-0.3.13-py3-none-any.whl (61 kB)
61.1/61.1 kB 5.5 MB/s eta 0:00:00
```

```
Installing collected packages: bert_score
Successfully installed bert_score-0.3.13
```

Import the scorer like this.

```
from bert_score import score as bert_score
```

 The cache for model files in Transformers v4.22.0 has been updated. Migrating your old cache. This is a one-time only operat
0/0 [00:00<?, ?it/s]

Your code should be written in the next code cell.

```
import torch
device = "cuda" if torch.cuda.is_available() else "cpu"
print(f"Using device: {device}")
bertscore_f1_scores = []
for i, generated_summary in enumerate(machine_summaries):
    P, R, F1 = bert_score(
        [generated_summary] * len(human_summaries[i]),
        human_summaries[i],
        lang="en",
        device=device,
        rescale_with_baseline=True,
    )
    bertscore_f1_scores.append(max(F1).item())
```

 [Show hidden output](#)

```
pearson_relevance, _ = pearsonr(bertscore_f1_scores, relevance_scores)
spearman_relevance, _ = spearmanr(bertscore_f1_scores, relevance_scores)

pearson_coherence, _ = pearsonr(bertscore_f1_scores, coherence_scores)
spearman_coherence, _ = spearmanr(bertscore_f1_scores, coherence_scores)

pearson_fluency, _ = pearsonr(bertscore_f1_scores, fluency_scores)
spearman_fluency, _ = spearmanr(bertscore_f1_scores, fluency_scores)

pearson_consistency, _ = pearsonr(bertscore_f1_scores, consistency_scores)
spearman_consistency, _ = spearmanr(bertscore_f1_scores, consistency_scores)

print("Pearson Correlations:")
```

```
print(f"Relevance: {pearson_relevance}")
print(f"Coherence: {pearson_coherence}")
print(f"Fluency: {pearson_fluency}")
print(f"Consistency: {pearson_consistency}")

print("\nSpearman Correlations:")
print(f"Relevance: {spearman_relevance}")
print(f"Coherence: {spearman_coherence}")
print(f"Fluency: {spearman_fluency}")
print(f"Consistency: {spearman_consistency}")
```

```
↗ Pearson Correlations:
Relevance: 0.3664939064033432
Coherence: 0.3852271128983417
Fluency: 0.16926907287618242
Consistency: 0.11277326972541596

Spearman Correlations:
Relevance: 0.3711227722935896
Coherence: 0.37691303618272515
Fluency: 0.1421129761645458
Consistency: 0.10858017111884058
```

In the next text cell, write a brief comment analyzing the results. Discuss any insights you observe, such as which human-authored scores correlate most strongly with BERTScore and possible reasons for these patterns. Additionally, compare BERTScore with ROUGE-2 and BLEU, highlighting differences and similarities based on how these metrics are defined.

Answer

We can see Relevance and Coherence have the highest correlation coefficient.

Similarity: We can see that for BERTScore, ROUGE-2 and BLEU have the highest correlation for Relevance.

Difference: BERTScore is better at capturing coherence because contextual embeddings encode sentence-level semantics. Poor at capturing coherence and fluency because they rely solely on token-level matches

✓ Part 5: BLEURT Evaluation and Correlation Analysis (20 points)

You will use another similarity-based measurement, [BLEURT](#), specifically [this implementation](#). Use the following commands to install it. For each generated summary:

1. Unlike other metrics, I think that this implementation does not accept a single generated summary and a list of human-authored summaries to return a score. Instead, pass a generated summary and each associated human-authored summary separately to the scorer, and return the maximum BLEURT score for that generated summary.
2. Record the resulting BLEURT in a list.

Using the list of BLEURT scores, compute the **Pearson** and **Spearman** correlation coefficients with each of the following human-authored scores:

- Relevance
- Coherence
- Fluency
- Consistency

Your final output should be **8 correlation coefficients**: 4 for Pearson and 4 for Spearman.

Note: This part will also be slower than ROUGE-2 and BLEU. Remember why? It will be faster if you connect to a GPU.

Use these commands to install the package.

```
!git clone https://github.com/google-research/bleurt.git
%cd bleurt
!pip install .
```

```
↗ Cloning into 'bleurt'...
remote: Enumerating objects: 134, done.
remote: Counting objects: 100% (18/18), done.
remote: Compressing objects: 100% (18/18), done.
remote: Total 134 (delta 0), reused 17 (delta 0), pack-reused 116 (from 1)
Receiving objects: 100% (134/134), 31.28 MiB | 37.86 MiB/s, done.
```

```

Resolving deltas: 100% (49/49), done.
/content/bleurt
Processing /content/bleurt
  Preparing metadata (setup.py) ... done
Requirement already satisfied: pandas in /usr/local/lib/python3.10/dist-packages (from BLEURT==0.0.2) (2.2.2)
Requirement already satisfied: numpy in /usr/local/lib/python3.10/dist-packages (from BLEURT==0.0.2) (1.26.4)
Requirement already satisfied: scipy in /usr/local/lib/python3.10/dist-packages (from BLEURT==0.0.2) (1.13.1)
Requirement already satisfied: tensorflow in /usr/local/lib/python3.10/dist-packages (from BLEURT==0.0.2) (2.17.1)
Requirement already satisfied: tf-slim>=1.1 in /usr/local/lib/python3.10/dist-packages (from BLEURT==0.0.2) (1.1.0)
Requirement already satisfied: sentencepiece in /usr/local/lib/python3.10/dist-packages (from BLEURT==0.0.2) (0.2.0)
Requirement already satisfied: absl-py>=0.2.2 in /usr/local/lib/python3.10/dist-packages (from tf-slim>=1.1->BLEURT==0.0.2)
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.10/dist-packages (from pandas->BLEURT==0.0.2)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.10/dist-packages (from pandas->BLEURT==0.0.2) (2024.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.10/dist-packages (from pandas->BLEURT==0.0.2) (2024)
Requirement already satisfied: astunparse>=1.6.0 in /usr/local/lib/python3.10/dist-packages (from tensorflow->BLEURT==0.0.2)
Requirement already satisfied: flatbuffers>=24.3.25 in /usr/local/lib/python3.10/dist-packages (from tensorflow->BLEURT==0.0.2)
Requirement already satisfied: gast!=0.5.0,!=0.5.1,!=0.5.2,>=0.2.1 in /usr/local/lib/python3.10/dist-packages (from tensorflow->BLEURT==0.0.2)
Requirement already satisfied: google-pasta>=0.1.1 in /usr/local/lib/python3.10/dist-packages (from tensorflow->BLEURT==0.0.2)
Requirement already satisfied: h5py>=3.10.0 in /usr/local/lib/python3.10/dist-packages (from tensorflow->BLEURT==0.0.2) (3.1)
Requirement already satisfied: libclang>=13.0.0 in /usr/local/lib/python3.10/dist-packages (from tensorflow->BLEURT==0.0.2)
Requirement already satisfied: ml-dtypes<0.5.0,>=0.3.1 in /usr/local/lib/python3.10/dist-packages (from tensorflow->BLEURT==0.0.2)
Requirement already satisfied: opt-einsum>=2.3.2 in /usr/local/lib/python3.10/dist-packages (from tensorflow->BLEURT==0.0.2)
Requirement already satisfied: packaging in /usr/local/lib/python3.10/dist-packages (from tensorflow->BLEURT==0.0.2) (24.2)
Requirement already satisfied: protobuf!=4.21.0,!=4.21.1,!=4.21.2,!=4.21.3,!=4.21.4,!=4.21.5,<5.0.0dev,>=3.20.3 in /usr/local/lib/python3.10/dist-packages (from tensorflow->BLEURT==0.0.2)
Requirement already satisfied: requests<3,>=2.21.0 in /usr/local/lib/python3.10/dist-packages (from tensorflow->BLEURT==0.0.2)
Requirement already satisfied: setuptools in /usr/local/lib/python3.10/dist-packages (from tensorflow->BLEURT==0.0.2) (75.1)
Requirement already satisfied: six>=1.12.0 in /usr/local/lib/python3.10/dist-packages (from tensorflow->BLEURT==0.0.2) (1.16)
Requirement already satisfied: termcolor>=1.1.0 in /usr/local/lib/python3.10/dist-packages (from tensorflow->BLEURT==0.0.2)
Requirement already satisfied: typing-extensions>=3.6.6 in /usr/local/lib/python3.10/dist-packages (from tensorflow->BLEURT==0.0.2)
Requirement already satisfied: wrapt>=1.11.0 in /usr/local/lib/python3.10/dist-packages (from tensorflow->BLEURT==0.0.2) (1)
Requirement already satisfied: grpcio<2.0,>=1.24.3 in /usr/local/lib/python3.10/dist-packages (from tensorflow->BLEURT==0.0.2)
Requirement already satisfied: tensorboard<2.18,>=2.17 in /usr/local/lib/python3.10/dist-packages (from tensorflow->BLEURT==0.0.2)
Requirement already satisfied: keras>=3.2.0 in /usr/local/lib/python3.10/dist-packages (from tensorflow->BLEURT==0.0.2) (3.5)
Requirement already satisfied: tensorflow-io-gcs-filesystem>=0.23.1 in /usr/local/lib/python3.10/dist-packages (from tensorflow->BLEURT==0.0.2)
Requirement already satisfied: wheel<1.0,>=0.23.0 in /usr/local/lib/python3.10/dist-packages (from astunparse>=1.6.0->tensorflow->BLEURT==0.0.2)
Requirement already satisfied: rich in /usr/local/lib/python3.10/dist-packages (from keras>=3.2.0->tensorflow->BLEURT==0.0.2)
Requirement already satisfied: namex in /usr/local/lib/python3.10/dist-packages (from keras>=3.2.0->tensorflow->BLEURT==0.0.2)
Requirement already satisfied: optree in /usr/local/lib/python3.10/dist-packages (from keras>=3.2.0->tensorflow->BLEURT==0.0.2)
Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.10/dist-packages (from requests<3,>=2.21.0->tensorflow->BLEURT==0.0.2)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.10/dist-packages (from requests<3,>=2.21.0->tensorflow->BLEURT==0.0.2)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.10/dist-packages (from requests<3,>=2.21.0->tensorflow->BLEURT==0.0.2)
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.10/dist-packages (from requests<3,>=2.21.0->tensorflow->BLEURT==0.0.2)
Requirement already satisfied: markdown>=2.6.8 in /usr/local/lib/python3.10/dist-packages (from tensorboard<2.18,>=2.17->tensorflow->BLEURT==0.0.2)
Requirement already satisfied: tensorboard-data-server<0.8.0,>=0.7.0 in /usr/local/lib/python3.10/dist-packages (from tensorboard<2.18,>=2.17->tensorflow->BLEURT==0.0.2)
Requirement already satisfied: werkzeug>=1.0.1 in /usr/local/lib/python3.10/dist-packages (from tensorboard<2.18,>=2.17->tensorflow->BLEURT==0.0.2)
Requirement already satisfied: MarkupSafe>=2.1.1 in /usr/local/lib/python3.10/dist-packages (from werkzeug>=1.0.1->tensorflow->BLEURT==0.0.2)
Requirement already satisfied: markdown-it-py>=2.2.0 in /usr/local/lib/python3.10/dist-packages (from rich->keras>=3.2.0->tensorflow->BLEURT==0.0.2)
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in /usr/local/lib/python3.10/dist-packages (from rich->keras>=3.2.0->tensorflow->BLEURT==0.0.2)
Requirement already satisfied: mdurl<=0.1 in /usr/local/lib/python3.10/dist-packages (from markdown-it-py>=2.2.0->rich->keras>=3.2.0->tensorflow->BLEURT==0.0.2)
Building wheels for collected packages: BLEURT
  Building wheel for BLEURT (setup.py) ... done
  Created wheel for BLEURT: filename=BLEURT-0.0.2-py3-none-any.whl size=16456764 sha256=fc5426364409f8e07e05568700f18c0fc727

```

Use this command to import the scorer.

```
from bleurt import score as bleurt_score
```

Your code should be written in the next code cell.

```

scorer = bleurt_score.BleurtScorer()
bleurt_scores = []
for i, generated_summary in enumerate(machine_summaries):
    scores = [scorer.score(references=[ref], candidates=[generated_summary])[0] for ref in human_summaries[i]]
    bleurt_scores.append(max(scores))

pearson_relevance, _ = pearsonr(bleurt_scores, relevance_scores)
spearman_relevance, _ = spearmanr(bleurt_scores, relevance_scores)

pearson_coherence, _ = pearsonr(bleurt_scores, coherence_scores)
spearman_coherence, _ = spearmanr(bleurt_scores, coherence_scores)

pearson_fluency, _ = pearsonr(bleurt_scores, fluency_scores)
spearman_fluency, _ = spearmanr(bleurt_scores, fluency_scores)

pearson_consistency, _ = pearsonr(bleurt_scores, consistency_scores)
spearman_consistency, _ = spearmanr(bleurt_scores, consistency_scores)

print("Pearson Correlations:")
print(f"Relevance: {pearson_relevance}")

```

```

print(f"Coherence: {pearson_coherence}")
print(f"Fluency: {pearson_fluency}")
print(f"Consistency: {pearson_consistency}")

print("\nSpearman Correlations:")
print(f"Relevance: {spearman_relevance}")
print(f"Coherence: {spearman_coherence}")
print(f"Fluency: {spearman_fluency}")
print(f"Consistency: {spearman_consistency}")

➡ Pearson Correlations:
Relevance: 0.33331863348795915
Coherence: 0.15400666311396186
Fluency: 0.17067243146524058
Consistency: 0.20109668539769232

Spearman Correlations:
Relevance: 0.30287813512420636
Coherence: 0.30287813512420636
Fluency: 0.10801868617003382
Consistency: 0.15736337411815346

```

In the next text cell, write a brief comment analyzing the results. Discuss any insights you observe, such as which human-authored scores correlate most strongly with BLEURT and possible reasons for these patterns. Additionally, compare BLEURT with BERTScore, ROUGE-2, and BLEU, highlighting differences and similarities based on how these metrics are defined.

Answer

Again we can see that Relevance has the highest correlation.

Similarity: Like BERTScore, BLEURT uses deep contextual embeddings, making it better at capturing semantic similarity than lexical metrics like ROUGE-2 and BLEU.

Difference:

- BLEU: Precision-weighted n-gram overlap with a brevity penalty.
- ROUGE-2: Recall-focused bigram overlap, optionally calculating F1-scores.
- BERTScore: Semantic similarity using contextual embeddings from pre-trained language models.
- BLEURT: Evaluates semantic similarity and factual correctness, leveraging fine-tuned BERT-based models.

✓ Bonus: Developing LLM-as-Judge and Correlation Analysis (7 points)

You can earn additional points by developing an **LLM-as-Judge**. Your starting point should be the prompts introduced in [Zhang et al. \(2023\)](#). You may choose to implement either **pairwise comparison** or **single answer grading**—the choice is yours.

You can use **closed LLMs** like ChatGPT or Claude, or **open models** available through Huggingface. Regardless of your choice, you must report the code you use to obtain the LLM's judgments, or if you choose to manually prompt Claude/ChatGPT, you must report your prompt.

As in earlier parts, compare the LLM-as-Judge's judgments to human scores for:

- Relevance
- Factuality
- Consistency
- Fluency

Note: Since APIs for closed models are not free, you may report correlations using only **30 generated instances**, but not less than that. Your task is to independently determine a reasonable approach for the LLM-as-Judge.

Tip:

- You will need to use another column from the dataset that you haven't used yet.

Your code/prompt should be written in the next code cell.

Start coding or [generate](#) with AI.

In the next text cell, write a brief comment analyzing the results. Discuss any insights you observe, such as which human-authored scores correlate most strongly with your LLM-as-Judge and possible reasons for these patterns. Additionally, compare your LLM-as-Judge with

BLEURT, BERTScore, ROUGE-2, and BLEU, highlighting differences and similarities based on how these metrics are defined.

Double-click (or enter) to edit

✓ Report of resources and prompts used

You must report all resources used in this part of the assignment. This includes specific prompts used for interacting with LLMs for purposes that comply with academic integrity and do not violate any standards of academic misconduct.

Double-click (or enter) to edit